

2.1 UNE TABLE = UN TABLEAU EXCEL DANS VOTRE ORDINATEUR

Chaque classe dans models.py est une table. Chaque attribut est une colonne. Chaque objet créé (instance) est une ligne.

EXEMPLE RÉEL – BOUTIQUE ERP :

```
class Produit(models.Model):
    nom = models.CharField(max_length=200)
    prix_vente = models.DecimalField(max_digits=10, decimal_places=0)
    stock = models.IntegerField(default=0)
    categorie = models.ForeignKey('Categorie', on_delete=models.SET_NULL, null=True)
```

Ce que ça donne visuellement dans la base de données :

id	nom	prix_vente	stock	categorie_id
1	Savon Palmolive	500	24	3
2	Huile Palme 1L	1200	12	1
3	Sucre 1kg	650	50	1

2.2 GUIDE DES TYPES DE CHAMPS – CHOISIR LE BON TYPE

TYPE DJANGO	QUAND L'UTILISER
CharField(max_length=N)	Texte court : nom, code, adresse
TextField()	Texte long : description, commentaire
IntegerField()	Nombre entier : quantité, âge, rang
DecimalField(max_d, d_place)	Prix, montants en FCFA
BooleanField()	Oui/Non : actif, payé, livré
DateField()	Date seule : date de naissance
DateTimeField()	Date + heure : timestamp de commande
ForeignKey(Modele, ...)	Lien vers une autre table (clé étranger)
ManyToManyField(Modele)	Plusieurs ↔ plusieurs : tags, rôles
ImageField(upload_to=...)	Photo : produit, profil
FileField(upload_to=...)	Document PDF, fichier Excel

2.3 CAS PRATIQUE – ADAPTER BOUTIQUE ERP EN PHARMACIE

CONTEXTE ORIGINAL (boutique générale) :

```
class Produit(models.Model):
    nom = models.CharField(max_length=200)
    prix_vente = models.DecimalField(max_digits=10, decimal_places=0)
    stock = models.IntegerField(default=0)
```

MODIFICATIONS POUR UNE PHARMACIE (ajouts uniquement, sans supprimer) :

```
class Produit(models.Model):
    nom = models.CharField(max_length=200)
    prix_vente = models.DecimalField(max_digits=10, decimal_places=0)
    stock = models.IntegerField(default=0)
    # --- AJOUTS PHARMACIE ---
    numero_lot = models.CharField(max_length=50, blank=True)
    date_peremption = models.DateField(null=True, blank=True)
    ordonnance_req = models.BooleanField(default=False)
    dosage = models.CharField(max_length=100, blank=True)
```

RÈGLE : Ne supprimez jamais un champ existant sans comprendre toutes les vues et templates qui l'utilisent. AJOUTEZ, ne retirez pas.

2.4 PROCÉDURE MIGRATION – OBLIGATOIRE APRÈS CHAQUE MODIFICATION

Après avoir modifié `models.py`, vous devez "migrier" la base de données.
Dans Gykhamine Studio, panneau de droite :

ÉTAPE 1 → Cliquer sur [makemigrations]
Le terminal doit répondre : "Migrations for 'boutique': ..."
Si erreur → relire votre code, il y a une faute de syntaxe.

ÉTAPE 2 → Cliquer sur [migrate]
Le terminal doit répondre : "Running migrations: OK"
Votre base de données est maintenant à jour.

SANS CES DEUX ÉTAPES, vos modifications n'existent pas en base.
C'est l'erreur numéro 1 des débutants.

2.5 ANALYSE DE TABLE – LIRE SES PROPRES DONNÉES

Dans un shell Python (terminal du Studio, commande "shell") :

```
# Voir tous les produits en stock critique (< 5 unités)
from boutique.models import Produit
critiques = Produit.objects.filter(stock__lt=5)
for p in critiques:
    print(f"{p.nom} – Stock restant : {p.stock}")

# Compter les produits par catégorie
from django.db.models import Count
stats = Produit.objects.values('categorie__nom').annotate(total=Count('id'))
for s in stats:
    print(f"{s['categorie__nom']} : {s['total']} produits")
```

Ces deux commandes constituent votre première "analyse de données"
intégrée – sans quitter le projet.

RÉSUMÉ DU CHAPITRE 2

- ✓ Classe Python → Table SQL → Tableau de données
 - ✓ Choisir le bon type de champ selon la nature de la donnée
 - ✓ Adaptation = ajouter des champs, rarement en supprimer
 - ✓ Toujours terminer par `makemigrations + migrate`
 - ✓ Shell Django = premier outil d'analyse de vos données
- SUITE : Chapitre 3 – Les vues (`views.py`) et la logique métier